PNP Formal Reconstruction Report

Compiled Lean theorem inventory and current claim boundary

Coordinate `PNP-FORMAL-RECONSTRUCTION-STATUS-2026-07-15-42`

The repository does not currently establish $P = NP$.

Public theorem emission is disabled. A direct finite raw-machine verifier now proves concrete CNF-SAT membership in NP with an explicit polynomial bound. It does not prove CNF-SAT in P, NP-hardness, NP-completeness, or $P = NP$. The general charged pipeline is linked to the raw machine kernel, and the Cook–Levin formula has an exact externally sized answer-independent slot schedule plus a direct fuelled specification cursor, but there is no raw polynomial-time formula builder or concrete CNFSAT reduction. The reviewed activation fingerprints are unset and no root theorem exists. The abstract string-handle bridge is never an activation source.

Compiled declarations	6800
Theorem-kind declarations	3077
Assumption-free theorem-kind declarations	2543
Project-specific axioms	4
Concrete publication gate passed	**false**

Inventory SHA-256: `af5a7560f15f77e988300dd90dcd7e1d2bb77d465c78900d30dd1791fa50b708`

Generated from the pinned compiled Lean environment, not from source-text parsing.

Contents

1 Current authority and claim boundary

This report is the current repository report surface. Its factual theorem inventory comes from the compiled `PNP` environment under `leanprover/lean4:v4.31.0`. The exporter enumerates `Environment.constants`, resolves each originating module, and calls `Lean.collectAxioms` for every exported project declaration. The committed inventory is `status/LEAN_THEOREM_INVENTORY.json`; the public mirror is byte-identical.

The target remains $P = NP$, but target wording—including the new inactive definition `PNP.Main.ConcretePEqualsNP`—is not theorem evidence. JavaScript acceptance, Boolean fields, JSON strings, historical release records, report prose, and external review cannot activate a theorem. Only the separately derived concrete publication gate may control theorem emission, and that gate is currently false.

The completed direct CNF verifier consumes the literal canonical pair of an encoded formula and assignment certificate. Lean proves universal acceptance equivalence, rejection equivalence, and absence of timeout at its explicit polynomial fuel bound, and constructs a proof-bearing `PolynomialTimeVerifier CNFSAT`. Thus `PNP.Concrete.CNFSAT` is in the concrete NP class. This is a verifier theorem, not a deterministic polynomial-time SAT algorithm.

The concrete Cook–Levin development proves that the generated CNF formula is semantically equivalent to ordinary raw verifier execution in both input modes. It now also bounds the actual canonical unary-indexed formula encoding by an explicit fixed-verifier polynomial evaluated only at external source-input length. That output-size theorem does not execute the formula builder as a raw finite machine, prove its construction runtime, or package a concrete polynomial reduction.

The new rectangular schedule allocates exact constraint, clause, token, and raw-bit opportunities without reading the already-materialized program, formula, token encoding, or encoded formula as schedule inputs. Filtering populated slots reproduces those canonical objects, and the raw-bit slot count is exactly the external encoded-size polynomial. This remains a pure specification: no theorem charges constant time per slot, implements a raw builder, or proves a construction-runtime bound.

The direct formula cursor decodes constraint, clause, token, and bit coordinates without first constructing those complete canonical lists. Its nested options distinguish out-of-range lookup from valid empty padding, and Lean proves exact prefix, full, one-step-short, terminal, and excess-fuel behavior. Filtering the exact full cursor output yields the canonical encoding. These are specification-level evaluation theorems, not a constant-time raw slot interpreter or raw builder.

Machine field	Current value
`mathematicalTheoremEstablished`	false
`publicTheoremEmissionAllowed`	false
`finalTheoremReady`	false
`publicTheoremStatement`	`null`
`rootLeanTheorem`	`PNP.Main.p_eq_np`
`rootLeanTheoremPresent`	false

2 Concrete publication gate

The compatibility entry is `PNP.Main.p_eq_np`; a matching name alone is insufficient. The separate publication target is `PNP.Main.ConcretePEqualsNP`. Eligibility also requires the exact theorem type, reviewed kernel fingerprints, and a tightly fixed Lean-standard axiom closure. The target is present as an inactive definition backed by finite charged-pipeline semantics; its presence is not a proof, and the compiler/refinement link to the raw machine kernel remains an explicit blocker for the general charged-pipeline target. The direct CNF verifier is one closed raw-machine instance; it does not supply that general link.

The expected fingerprints are intentionally unset. This is a fail-closed migration gate, not a missing runtime input. A verifier must reject any attempt to set `passed=true` while a fingerprint is null, while the concrete model is ineligible, or while any other subcheck is false.

Gate subcheck	Result
standardComplexityModelEligible	**true**
concreteTargetPresent	**true**
concreteTargetIsDefinition	**true**
concreteTargetKernelTypeFingerprintConfigured	**false**
concreteTargetKernelTypeFingerprintMatches	**false**
concreteTargetKernelValueFingerprintConfigured	**false**
concreteTargetKernelValueFingerprintMatches	**false**
compatibilityRootPresent	**false**
compatibilityRootIsTheorem	**false**
compatibilityRootHasExactConcreteType	**false**
compatibilityRootKernelTypeFingerprintConfigured	**false**
compatibilityRootKernelTypeFingerprintMatches	**false**
axiomClosureFingerprintConfigured	**false**
axiomClosureFingerprintMatches	**false**
sourceClosureFingerprintConfigured	**false**
sourceClosureFingerprintMatches	**false**
axiomClosureUsesOnlyLeanStandardAllowlist	**false**

Allowed foundation axioms are fixed to `Classical.choice`, `Quot.sound`, and `propext`. Project axioms, `sorryAx`, or any unknown axiom make the gate fail. The four currently disclosed project axioms are:

- `PNP.CheckPCCPackexp`
- `PNP.GeneratePCCPack`
- `PNP.LockedNANDThreshold`
- `PNP.ResidualBandExactMinimization`

2.1 Why the abstract bridge is ineligible

The current `PNP.PEqualsNP` abbreviation compares witness classes whose language and algorithm structures carry string names or code handles rather than concrete execution semantics and proved polynomial bounds. Consequently, even a kernel-clean theorem of that abstract type would not establish the standard P-versus-NP statement. It is compatibility information only.

The separate `PNP.Concrete.PEqualsNP` target uses proof-bearing P and NP witnesses, canonical input/certificate pairing, finite program syntax, and charged polynomial runtime and output bounds. This is a substantive formalization milestone, but it is not yet proved equivalent to execution in the raw machine kernel. SAT completeness, SAT in P, and the root theorem also remain absent.

3 Formal milestone ledger

Each earned row requires exact compiled names and theorem kinds, empty axiom closures, per-name domain-separated kernel-type SHA-256 matches for 299 detailed candidates, and the pinned whole-Lean source closure. Human-readable scope remains conservative: type or source drift revokes credit.

Milestone	Status	Exact scope	Boundary
Concrete machine and cost kernel	formalized-foundation-only	One literal finite four-stage pipeline handles every raw bitstring. The total framer uses exactly 4 work steps on empty input, 4 * k * k + 9 * k + 7 for complete two-bit cells, and 4 * k * k + 9 * k + 5 when the final cell is partial; its compiled raw cost is bounded by 6 * m * m + 39 * m + 75. Every supplied exact n-step target run lifts to exactly 3 * n work steps. PipelineCompiler preserves one raw target's verdict and ordinary output at an explicit external polynomial. PipelineSequentialCompiler composes two raw targets in one literal finite table, passes either first verdict onward, preserves the second verdict and output, retains stuck-first timeout, and proves R(m) = PipelineRaw(p)(m) + 6 + PipelineRaw(q)(m + p(m) + 1). PipelineRefinement recursively applies that compiler to every FunctionProgram composition and DecisionProgram precomposition node. Its 16 audited declarations have empty axiom closure, and PolynomialTimeDecider.compileToMachine exposes the complete tree as one raw polynomial-time machine.	This closes the concrete complexity machine-link blocker only. It does not construct a deterministic polynomial-time CNF-SAT decider or an NP-completeness reduction. CNF-SAT in P, NP-completeness, and P = NP are not established; PNP.Main.p_eq_np remains absent and the publication gate remains false.
Concrete P, NP, and polynomial reductions	formalized	Bitstring languages, finite charged decision/verifier/function pipelines grounded in concrete machines, polynomial certificate/runtime/output bounds, recursively compiled exact raw-machine refinements, many-one reductions, and the NP-complete-in-P implication.	The recursive refinement compiler closes the charged-to-raw machine link, but it does not construct raw paired verifiers beyond the existing CNF-SAT membership witness, prove CNF-SAT is in P or NP-complete, or establish the publication root theorem.

Milestone	Status	Exact scope	Boundary
Concrete universal CNF-SAT verifier	formalized-np-membership-only	An unconditional formula-grammar outcome, universal bounded work outcome, exact accept/reject semantics, no-timeout theorem, literal finite-machine paired verifier, and CNF-SAT membership in NP.	This proves CNF-SAT membership in NP only; it does not prove CNF-SAT is in P, NP-completeness, or P = NP.
Concrete Cook-Levin dimensions and variable layout	formalized-foundation-only	Fuel padding preserves designated halts and stuck timeouts; every literal machine state is below an explicit finite ceiling; the input, time, and tape dimensions are executable NatPolynomial expressions; and tape-symbol, head, state, certificate-bit, and certificate-length variables occupy proved disjoint in-range blocks. Elementary at-least-one and pair-exclusion CNF clauses have exact propositional semantics. All 20 reviewed theorems have empty axiom closure.	This is the finite layout substrate only. It does not yet construct a transition tableau, prove tableau soundness or completeness, compile a formula emitter, define a polynomial reduction to CNFSAT, or establish CNFSAT NP-completeness, CNFSAT in P, or P = NP.
Concrete fixed-certificate execution tableau	formalized-foundation-only	For one fixed source input, certificate, finite-rule machine, and fuel budget, the canonical raw execution trace has exactly fuel + 1 configurations. Intrinsic first-match transition validity is equivalent to equality with that trace, every valid endpoint equals the ordinary run endpoint, and accept/reject/timeout verdicts agree exactly with boundedDecide. All 30 declarations and eight reviewed theorem types have empty axiom closure.	This is the semantic fixed-certificate tableau only. It does not yet quantify a variable certificate, encode tape windows or tableau rows as CNF, compile a formula emitter, define a polynomial reduction to CNFSAT, or establish CNFSAT NP-completeness, CNFSAT in P, or P = NP.

Milestone	Status	Exact scope	Boundary
Uniform bounded verifier tableaux	formalized-foundation-only	For an arbitrary proof-bearing verifier in the concrete NP model and one source input, the complete finite decision pipeline is compiled to one raw machine. Every certificate inside the explicit certificate polynomial shares one answer-independent raw fuel obtained by evaluating the compiled raw-time polynomial at the maximum encoded input size. Per-certificate fuel is below that bound, padding follows a proved halt, exact verdicts are preserved, and language membership is equivalent to an existential bounded accepting semantic tableau. All 41 declarations and nine reviewed theorem types have empty axiom closure.	This is a uniform semantic verifier-tableau equivalence only. It does not encode configurations, transition windows, or variable-length certificates as Boolean clauses; emit CNF; prove emitter size or runtime polynomials; define a polynomial reduction to CNFSAT; or establish CNFSAT NP-completeness, CNFSAT in P, or P = NP.
Finite local Cook-Levin CNF compiler	formalized-foundation-only	Width-indexed literals materialize exact Boolean assignments and compile to the concrete CNF semantics without out-of-range variables. Unit constraints, finite implications, and pairwise exactly-one groups have exact satisfaction equivalences; local programs have exact recursive clause counts, satisfiability reflection, and a proved well-scoped formula. All 75 declarations and nine reviewed theorem types have empty axiom closure.	This is a local clause compiler only. It does not enumerate the verifier tableau, encode initialization, transition windows, certificate length, or acceptance as a complete formula; prove an all-input emitter size/runtime polynomial; define a reduction to CNFSAT; or establish CNFSAT NP-completeness, CNFSAT in P, or P = NP.

Milestone	Status	Exact scope	Boundary
Finite whole-tableau Cook-Levin CNF syntax	formalized-foundation-only	A uniform bounded verifier-tableau problem is compiled answer-independently into one finite, well-scoped concrete CNF formula. The program emits one-hot symbol/head/state rows, first-match control updates, untouched-cell preservation, exact input-only or symbolic paired-certificate initialization, and a designated final accept constraint. Canonical encoding/decoding, exact local satisfaction reflection, and exact emitted clause counting are proved. All 81 declarations and ten reviewed theorem types have empty axiom closure.	This is finite formula syntax and local-clause reflection only. It does not yet prove that formula satisfiability is equivalent to an accepting raw verifier execution, prove external encoded-output-size or construction-runtime polynomials, define a concrete polynomial reduction to CNFSAT, or establish CNFSAT NP-completeness, CNFSAT in P, or P = NP.
Finite whole-tableau Cook-Levin CNF semantics	formalized-foundation-only	Every satisfying assignment decodes to a unique-symbol, unique-head, unique-state finite row at each bounded time. The decoded initial row is exact in both verifier modes, each adjacent row is the literal deterministic first-match successor, and the final state is accepting. Conversely, every such intrinsic finite accepting tableau constructs a satisfying assignment. Formula satisfiability and concrete CNFSAT membership are therefore equivalent to this finite semantics. All 161 explicit declarations are axiom-audited; the six reviewed theorem types depend only on the permitted Lean standard axioms Quot.sound and propext, with no project axiom.	This milestone itself proves equivalence only with an intrinsic finite-row model; the following raw-tape milestone supplies the execution bridge. External encoded-output-size and construction-runtime polynomials, a concrete polynomial reduction to CNFSAT, CNFSAT NP-completeness, CNFSAT in P, and P = NP remain absent.

Milestone	Status	Exact scope	Boundary
Finite tableau to raw Tape semantics	formalized-foundation-only	The finite Cook-Levin rows are proved exactly equivalent to ordinary focused raw Tape execution. Absolute two-sided tape observations cover explicit and implicit blanks; the local successor matches designated halts, missing-rule stutter, and the literal first matching raw rule; a centered head invariant rules out finite-window clamping; and both verifier input modes have exact initial rows, including reversible bounded-certificate reconstruction. Formula satisfiability and concrete CNFSAT membership are therefore equivalent to the concrete verifier language. All 54 explicit declarations are axiom-audited, and the nine reviewed theorem types use only the permitted Lean standard axiom allowlist with no project axiom.	This proves exact semantic reduction correctness only. It does not yet prove external encoded-formula-size or formula-construction-runtime polynomials, package a concrete PolynomialReduction, establish CNFSAT NP-completeness, CNFSAT in P, or P = NP.
External Cook-Levin encoded-formula size	formalized-foundation-only	The actual canonical unary-indexed CNF bitstring generated for a fixed concrete polynomial-time verifier is bounded by an explicit NatPolynomial evaluated only at the external source-input length. Exact codec lengths, both verifier input modes, every concrete tableau constraint family, program and emitted-clause counts, and clause width are covered. All 108 explicit declarations are axiom-audited; the nine reviewed theorem types use only the permitted Lean standard axioms Quot.sound and propext, with no project axiom or choice axiom.	This does not implement or time a raw finite formula builder, package a concrete PolynomialReduction, establish CNFSAT NP-completeness, establish CNFSAT in P, or prove P = NP.

Milestone	Status	Exact scope	Boundary
Rectangular Cook-Levin formula schedule	formalized-foundation-only	For every concrete verifier-tableau problem, a finite answer-independent rectangular schedule allocates exact constraint, clause, token, and raw-bit slots. Removing empty slots in order reproduces the existing canonical local program, bounded clauses, CNF tokens, and encoded formula. The raw-bit schedule length is exactly the previously proved encodedFormulaSizePolynomial evaluated at external source-input length. All 79 explicit declarations are axiom-audited; the eight reviewed theorem types use only the permitted Lean standard axioms Quot.sound and propext, with no project axiom or choice axiom.	This is a pure schedule specification. It does not interpret a slot as a constant-time raw-machine action, implement or time a raw finite formula builder, construct a FunctionProgram.RawRefinement, package a concrete PolynomialReduction, establish CNFSAT NP-completeness, establish CNFSAT in P, or prove P = NP.
Direct Cook-Levin formula cursor	formalized-foundation-only	For every concrete verifier-tableau problem and natural coordinate, direct constraint, clause, token, and raw-bit decoders agree pointwise with the canonical rectangular schedules while preserving out-of-range, valid-padding, and populated states. A fuelled specification cursor proves exact bounded prefixes, complete traversal, one-step-short behavior, terminal behavior, excess-fuel stability, the external encoded-size polynomial, and canonical emitted output. All 129 explicit declarations are axiom-audited; the 13 reviewed theorem types use only the permitted Lean standard axioms Quot.sound and propext, with no project axiom or choice axiom.	This is a Lean specification cursor. It does not prove constant-time raw slot interpretation, implement or time a raw finite formula builder, construct a FunctionProgram.RawRefinement, package a concrete PolynomialReduction, establish CNFSAT NP-completeness, establish CNFSAT in P, or prove P = NP.
Typed direct-wire NAND semantics	formalized	Typed topological Boolean NAND programs and ordered multi-output direct-wire semantics.	This does not establish circuit minimization, SAT, or P = NP.

Milestone	Status	Exact scope	Boundary
Finite enumeration, equivalence, and reference minimum	formalized	Exhaustive finite Boolean direct-wire search under the empty-profile reference model.	No polynomial-runtime result is obtained from this exhaustive reference search.
Concrete framed replacement and slack	formalized	The concrete serial framed context with support outputs and explicit bypass wires.	This is not an arbitrary-support replacement theorem for the report's global family.
Locked-NAND local candidates and baseline accounting	formalized	Typed local macro candidates, source-derived counts, and five finite local square baselines.	Local minima are not a global BaselineDistinct or locked-NAND threshold theorem.
Conditional locked-NAND threshold boundary	formalized-with-premises	A proof-bearing six-premise candidate boundary for an arbitrary satisfiable proposition.	The premises are not instantiated by a uniform SAT-to-locked-NAND builder.
Fail-closed explicit-list residual routes	formalized-explicit-list-only	Executable strict-gain search over a caller-supplied finite implementation list.	Unresolved excludes no gain outside the supplied list and cannot imply ZeroSlack.
Global locked-NAND construction and threshold	not-formalized	A uniform answer-independent SAT instance builder, global baseline, trace equivalence, and threshold.	The conditional boundary does not discharge this missing theorem.
Global ZeroSlack, PCCMin, and polynomial runtime	not-formalized	Complete residual routing, global ZeroSlack contradiction, exact minimization, and polynomial bounds.	Proof-bearing result structures and explicit-list failure do not supply global completeness.
Concrete standard P-versus-NP target and root theorem	not-formalized	Raw-machine-linked complexity classes, concrete SAT completeness and a SAT decider, plus the publication root theorem.	The concrete target definition remains inactive: CNF-SAT now has a raw-machine verifier and NP-membership proof, but a deterministic polynomial-time CNF-SAT decider, NP-completeness transport, and PNP.Main.p_eq_np remain absent; the legacy string-handle bridge is ineligible.

4 Compiled inventory summary

The inventory contains 6800 exported `PNP.*` kernel declarations from 61 modules. It excludes 1128 private compiler auxiliaries. Counts describe the compiled environment; they do not measure mathematical completeness.

Module	Declarations	Theorems
PNP.Bridge	93	22
PNP.Complexity	93	20
PNP.Concrete.BitString	123	60
PNP.Concrete.CNF	247	60
PNP.Concrete.CNFVerifier	10	7

Module	Declarations	Theorems
PNP.Concrete.CNFWorkCorrectness	859	525
PNP.Concrete.CNFWorkFrameCorrectness	16	4
PNP.Concrete.CNFWorkInput	29	14
PNP.Concrete.CNFWorkMachine	85	3
PNP.Concrete.CNFWorkTransitions	64	63
PNP.Concrete.CNFWorkUniversalCorrectness	294	145
PNP.Concrete.Complexity	312	92
PNP.Concrete.CookLevinFormulaCursor	229	113
PNP.Concrete.CookLevinFormulaSchedule	112	81
PNP.Concrete.CookLevinFormulaSize	153	136
PNP.Concrete.CookLevinLayout	204	71
PNP.Concrete.CookLevinLocalCNF	172	53
PNP.Concrete.CookLevinRawTapeBridge	109	97
PNP.Concrete.CookLevinTableau	71	23
PNP.Concrete.CookLevinTableauCNF	179	60
PNP.Concrete.CookLevinTableauCNFSemantics	190	136
PNP.Concrete.CookLevinVerifierTableau	58	25
PNP.Concrete.Machine	284	67
PNP.Concrete.PipelineCompiler	38	27
PNP.Concrete.PipelineInputFramer	106	18
PNP.Concrete.PipelineMachineSimulation	175	79
PNP.Concrete.PipelineOutputHandoff	23	4
PNP.Concrete.PipelinePairedCompiler	29	25
PNP.Concrete.PipelineRefinement	68	24
PNP.Concrete.PipelineSequentialCompiler	36	28
PNP.Concrete.PipelineSequentialStateNamespace	30	21
PNP.Concrete.PipelineStageBridges	77	59
PNP.Concrete.PipelineStateNamespace	77	34
PNP.Concrete.PipelineTapeGeometry	20	12
PNP.Concrete.PipelineTerminalBridge	73	62
PNP.Concrete.TapeBlankEquivalence	25	17
PNP.Concrete.TapeHandoff	18	11
PNP.Concrete.Target	2	1
PNP.Concrete.TerminalOutputPacker	111	36
PNP.Concrete.WorkInput	23	14
PNP.Concrete.WorkMachine	308	108
PNP.DirectWireBaseline	48	25
PNP.LockedNAND	11	4
PNP.LockedNANDBaseline	57	22
PNP.LockedNANDDirect	84	57
PNP.LockedNANDLocalBaseline	30	22
PNP.LockedNANDMacros	288	98
PNP.LockedNANDPrefix	84	40
PNP.LockedNANDThresholdBoundary	60	35
PNP.Main	36	12
PNP.NANDComposition	77	38
PNP.NANDEnumerator	120	46
PNP.NANDMinimum	56	23
PNP.NANDSemantics	198	80
PNP.NANDSlack	15	12
PNP.NANDTruthTable	72	28
PNP.PCCMin	44	8
PNP.ResidualBand	9	2
PNP.ResidualRoutes	141	46
PNP.SAT	4	1
PNP.ZeroSlack	141	21

5 Remaining formal blockers

Seven formal blockers remain active:

1. `Formal.ConcreteSAT`
2. `Formal.LockedNANDThreshold`
3. `Formal.ResidualBandMinimizer`

4. `Formal.ZeroSlack`
5. `Formal.PolynomialRuntimeAndCertificateBounds`
6. `Formal.RootTheoremAndAxiomAudit`

The conditional locked-NAND threshold module assumes typed baseline and full candidates, baseline conditions, preservation of existing outputs, an unsatisfiable final-zero law, and satisfiable final-output laws. The explicit residual scanner searches only a caller-supplied finite list. Neither result constructs the missing global reduction or proves global ZeroSlack, polynomial PCCMin, SAT in P, or P equals NP.

6 Historical report provenance

Earlier 56-page report revisions stated a direct checker-mediated P-versus-NP claim. Those bytes remain historical audit material at their pinned legacy coordinates and through the explicit archive and supersession records. They are not imported into, quoted as authority by, or used to generate this report. File identity and replay of historical predicates do not establish the named mathematical propositions.

Source tag	`final-pnp-proof-report-hardened-7072f8d`
Source commit	`7072f8d0bda6d44d240f9bb3fad624fd357e1278`
Archive manifest	`archive/legacy-v0/ARCHIVE.json`

7 Verification

The current reconstruction can be checked with:

```
lake build PNP
node scripts/export-lean-theorem-inventory.mjs --check
node scripts/generate-formal-publication.mjs --check
node pcc-formal-reconstruction-status0.mjs --json
npm run pnp:verify -- --no-write
npm run report:check
```

The inventory coordinate is `PNP-LEAN-THEOREM-INVENTORY-2026-07-15-42`. Its exact byte digest is `af5a7560f15f77e988300dd90dcd7e1d2bb77d465c78900d30dd1791fa50b708`. The report is regenerated from that inventory and the reviewed publication map. The reviewed milestone source-closure SHA-256 is `d88b76196a15dba8be17ce2cde2612c36391efe1df03243687ce5005c8a71c5e`; the computed and pinned values match. Successful regeneration confirms consistency of these status surfaces; it does not turn an absent concrete theorem into a proof.